

UNIVERSIDAD METROPOLITANA UNIDAD CUAJIMALPA



**UNIVERSIDAD
AUTÓNOMA
METROPOLITANA**
Unidad Cuajimalpa

Trimestre 26-P

SEMINARIO DE TECNOLOGIAS DE LA INFORMACION II

Proyecto - Entrega 2 Enjambre

Profesor: Issac Rudomin

Alumno:

- **Javier Arturo Cirilo
Vargas 2233030596**
- **Camila Anaid Cruz
Rodriguez 2233070563**
- **Julio Cesar Aguirre
Crescencio 2233030292**

1. Introducción

El objetivo de esta entrega es ampliar la primera versión del enjambre integrando los conocimientos de los temas T1, T2, T4 y T5. Ahora el sistema muestra cientos de agentes animados con un shader propio y animaciones independientes, todo corriendo en tiempo real dentro del navegador usando Three.js y WebGL.

2. Objetivo

Renderizar un enjambre de al menos 100 agentes en tiempo real usando InstancedMesh (T5) con animación por Vertex Animation Texture (VAT), aplicando al menos un shader avanzado de T4 (Toon/Cel-shading o Vertex Wobble), manteniendo un rendimiento mínimo de 30 FPS y mostrando métricas en pantalla.

3. Requisitos cumplidos

A continuación se muestra una tabla con los requisitos de la entrega y la manera en que cada uno fue implementado:

Requisito	Implementación
≥ 100 agentes con InstancedMesh	Slider de 50–500 agentes (default 100). Un solo InstancedMesh representa a todos.
VAT / boneVAT (T5)	DataTexture RGBA Float32 de VERT_COUNT × 64 frames. El vertex shader samplea con gl_VertexID y un aTimeOffset por instancia.
Shader avanzado T4	Toon/Cel-shading con bandas cuantizadas y contorno back-face inflate (outline oscuro).
Shader adicional T4	Vertex Wobble: deformación senoidal $A \cdot \sin(\text{freq} \cdot \text{pos.y} - \text{speed} \cdot \text{time})$ en el vertex shader.

Movimiento coherente	Movimiento orbital con alineación suave de velocidad (flocking parcial T2). Determinístico con sin/cos y phase offset individual.
30 FPS con 100 agentes	1 draw call para todos los agentes gracias a InstancedMesh. Validado en el HUD.
FPS en pantalla	HUD con FPS, Draw Calls, Triángulos y Shader activo en tiempo real.

4. Arquitectura técnica

4.1 T5 — InstancedMesh + VAT

Todos los agentes del enjambre se representan con un único InstancedMesh de Three.js, lo que reduce a una sola draw call el costo de enviar geometría a la GPU. Esto es fundamental para alcanzar el objetivo de rendimiento.

La animación de caminar de cada agente se almacena en una Vertex Animation Texture (VAT): una DataTexture de tipo RGBA Float32 con dimensiones $VERT_COUNT \times 64$ frames. Cada texel guarda el offset de posición (offsetX, offsetY, offsetZ) y un valor de squash. El vertex shader samplea la textura usando `gl_VertexID` y cada instancia recibe un `aTimeOffset` diferente para desincronizar las animaciones.

4.2 T4 — Toon / Cel-shading

El shader de Toon/Cel-shading simula el estilo visual de los dibujos animados. En lugar de una iluminación difusa continua, se cuantiza el valor de luz usando la expresión $\text{floor}(\text{diff} * \text{bands}) / \text{bands}$, lo que produce transiciones abruptas entre zonas iluminadas y zonas en sombra. El especular también es discreto, calculado con `step()`. El contorno oscuro (outline) se genera con un segundo InstancedMesh renderizado con `side: BackSide` que infla los vértices en dirección a la normal (`back-face inflate`).

4.3 T4 — Vertex Wobble

El segundo shader avanzado es el Vertex Wobble. Aplica una deformación senoidal sobre la posición de cada vértice de la forma $A \cdot \sin(\text{freq} \cdot \text{pos.y} - \text{speed} \cdot \text{time})$. La deformación se suma a la posición ya animada por el VAT, dando una apariencia de movimiento fluido y orgánico.

4.4 T1 + T2 — Movimiento del enjambre

Cada agente sigue una trayectoria orbital con un phase offset individual generado con funciones sin/cos. El movimiento es determinístico y escala fácilmente a cientos de agentes. Se aplica además una alineación suave de velocidad (cohesión), que es la base del flocking completo que se integrará en la entrega final con T6.

5. Controles de la demo

La aplicación incluye un panel de controles en tiempo real en la esquina superior derecha de la pantalla:

Control	Descripción
Agentes	Slider de 50 a 500. Reconstruye el InstancedMesh al mover el slider.
Shader	Cambia entre Toon, Vertex Wobble y Normal debug.
Bandas Toon	Cuantización de luz (2–8 bandas). Solo activo en modo Toon.
Grosor contorno	Ajusta el inflado de normales para el outline oscuro.
Amplitud wobble	Controla la intensidad de la deformación senoidal.
Velocidad swarm	Multiplicador de velocidad global del enjambre.
VAT frames	Velocidad de reproducción de la animación horneada.

Contorno	Toggle para activar o desactivar el outline oscuro.
Cámara orbital	Activa o desactiva la rotación automática de la cámara.

6. Instrucciones de ejecución

La demo requiere un servidor HTTP local porque los shaders necesitan hacer peticiones de recursos y el protocolo file:// genera errores de CORS. Las opciones disponibles son:

1. Python: ejecutar `python -m http.server 8080` dentro de la carpeta y abrir `http://localhost:8080`.
2. Node.js: ejecutar `npx serve .` en la carpeta del proyecto.
3. VS Code: click derecho en `index.html` → "Open with Live Server".

7. Conclusión

La segunda entrega del proyecto Enjambre cumple con todos los requisitos establecidos. Se logró renderizar 100 agentes en tiempo real manteniendo más de 30 FPS gracias al uso de `InstancedMesh`, que reduce todo el enjambre a una sola draw call. La animación individual por VAT permite que cada agente tenga una animación desincronizada sin añadir costo adicional en CPU.

Se implementaron dos shaders avanzados de T4: el Toon/Cel-shading, que da al enjambre un estilo visual llamativo con contornos definidos, y el Vertex Wobble, que añade movimiento orgánico a los agentes. El sistema queda preparado para integrar el flocking completo en la entrega final con T6.

8. Link del repositorio de GitHub

<https://github.com/Camila23370/Proyecto-STI-Enjambre-Camilos.git>